



Методы шифрования и их реализация

Защита информации

Дальневосточный государственный университет путей сообщения
(ДВГУПС)

38 pag.

Министерство транспорта Российской Федерации
Федеральное агентство железнодорожного транспорта
Федеральное государственное бюджетное образовательное учреждение высшего образования
Институт интегрированных форм обучения
«Дальневосточный государственный университет путей сообщения»

Контрольная работа

по дисциплине «Защита информации»

Выполнил _____ Рустамов Ф.Т.

Шифр _____ КВ16-ИВТ(Б)-006

Проверил _____ Данилова Е.В.

Криптосхемы классической криптографии

2. Шифр Цезаря

Данный шифр был придуман Гаем Юлием Цезарем и использовался им в своей переписке (1 век до н.э.). Применительно к русскому языку суть его состоит в следующем. Выписывается исходный алфавит (А, Б, ..., Я), затем под ним выписывается тот же алфавит, но с циклическим сдвигом на 3 буквы влево.

А	Б	В	Г	Д	Е	Ж	З	И	Й	К	Л	М	Н	О	П	Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ы	Ь	Ъ	Э	Ю	Я
Г	Д	Е	Ж	З	И	Й	К	Л	М	Н	О	П	Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ы	Ь	Ъ	Э	Ю	Я	А	Б	В

При зашифровке буква А заменяется буквой Г, Б - на Д и т. д. Так, например, исходное сообщение «АБРАМОВ» после шифрования будет выглядеть «ГДУГПСЕ». Получатель сообщения «ГДУГПСЕ» ищет эти буквы в нижней строке и по буквам над ними восстанавливает исходное сообщение «АБРАМОВ».

Ключом в шифре Цезаря является величина сдвига нижней строки алфавита. Количество ключей для всех модификаций данного шифра применительно к алфавиту русского языка равно 33. Возможны различные модификации шифра Цезаря, в частности лозунговый шифр.

Скриншоты работы программы

Рисунок SEQ Рисунок * ARABIC 1 - шифр Цезаря (1)

Рисунок SEQ Рисунок * ARABIC 2 - шифр Цезаря(2)

Код программы

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Text;
using System.Windows.Forms;
namespace _1_1.Шифр_Цезаря_сдвига_
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();

            // шифр сдвига
            #region
            private void button1_Click(object sender, EventArgs e)
            {
                txtOut.Text = CaesarCipher.Decryption(txtIn.Text);
            }

            private void button2_Click(object sender, EventArgs e)
            {
                txtOut.Text = CaesarCipher.Encryption(txtIn.Text);
            }

            #endregion
        }

        // реализация ЦЕзаря (шифратор + дешифратор)
        class CaesarCipher
        {
            static string alph = "абвгдеёжзийклмнопрстуфхцчщъыьэюя";

            public static string Encryption(string text)
            {
                var res = new StringBuilder();
                4
            }
        }
    }
}
```

```

        for (int i = 0; i < text.Length; i++)
            for (int j = 0; j < alph.Length; j++)
                if (text[i] == alph[j]) res.Append(alph[(j + 3) %
alph.Length]);

        return res.ToString();
    }
    public static string Decryption(string crypt)
    {
        var res = new StringBuilder();
        for (int i = 0; i < crypt.Length; i++)
            for (int j = 0; j < alph.Length; j++)
                if (crypt[i] == alph[j]) res.Append(alph[(j - 3 + alph.Length)
% alph.Length]);

        return res.ToString();
    }
}
}

```

3. Шифр подстановки

Для реализации шифра подстановки нужен алфавит, который будем хранить в key.txt, и сама подстановка, которую будем брать из def.txt. Каждая строка из первого файла будет заменяться на соответствующую строку из второго.

Например:

Key.txt	Def.txt
а	1
б	2
в	3

Скриншоты работы программы

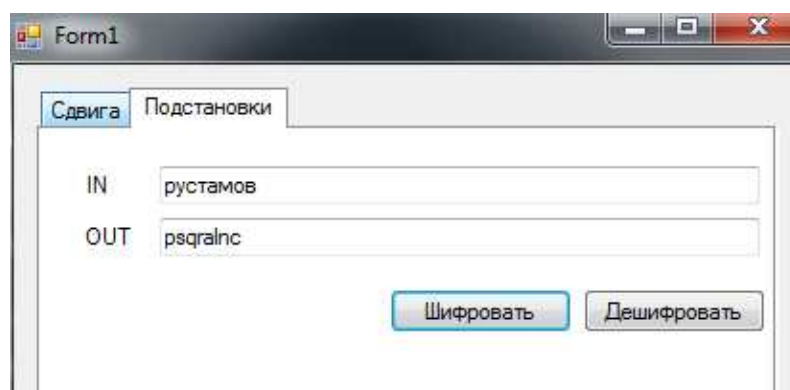


Рисунок SEQ Рисунок * ARABIC 3 - шифр подстановки(1)

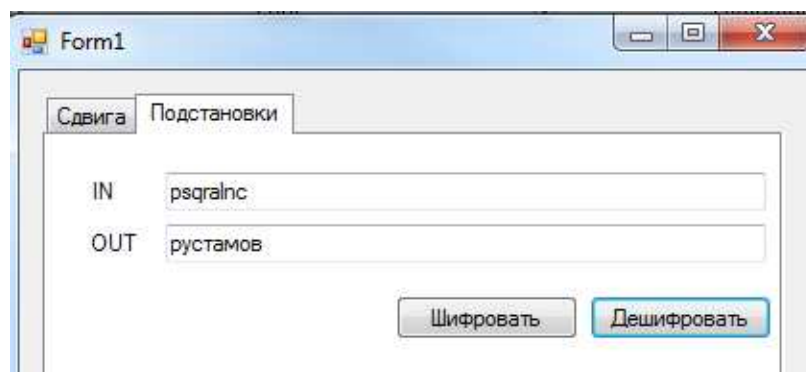


Рисунок SEQ Рисунок * ARABIC 4 - шифр подстановки(2)

Код программы

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Text;
using System.Windows.Forms;
namespace _1_1.Шифр_Цезаря_сдвига_
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();

            // шифр подстановки
            #region
            private void button3_Click(object sender, EventArgs e)
            {
                string[] alf = File.ReadAllLines("replace_orig.txt",
Encoding.GetEncoding(1251));
                string[] key = File.ReadAllLines("replace_rep.txt",
Encoding.GetEncoding(1251));
                //string text = File.ReadAllText("text.txt",
Encoding.GetEncoding(1251));
                string text = textBox1.Text;

                Dictionary<string, string> substitution = new Dictionary<string,
string>();

                for (int i = 0; i < alf.Length; i++) { substitution[alf[i]] =
key[i]; }

                StringBuilder ans = new StringBuilder(); for (int i = 0; i <
text.Length; i++) { if (substitution.ContainsKey(text[i].ToString()))
{ ans.Append(substitution[text[i].ToString()]); } else { ans.Append(text[i]); } }
                //File.WriteAllText("code.txt", ans.ToString(),
Encoding.GetEncoding(1251));
                textBox2.Text = ans.ToString();
            }

            private void button4_Click(object sender, EventArgs e)
```



```

    {
        string[] alf = File.ReadAllLines("replace_rep.txt",
Encoding.GetEncoding(1251));
        string[] key = File.ReadAllLines("replace_orig.txt",
Encoding.GetEncoding(1251));
        //string text = File.ReadAllText("text.txt",
Encoding.GetEncoding(1251));
        string text = textBox1.Text;

        Dictionary<string, string> substitution = new Dictionary<string,
string>();

        for (int i = 0; i < alf.Length; i++) { substitution[alf[i]] =
key[i]; }

        StringBuilder ans = new StringBuilder(); for (int i = 0; i <
text.Length; i++) { if (substitution.ContainsKey(text[i].ToString()))
{ ans.Append(substitution[text[i].ToString()]); } else { ans.Append(text[i]); } }

        //File.WriteAllText("code.txt", ans.ToString(),
Encoding.GetEncoding(1251));
        textBox2.Text = ans.ToString();
    }
    #endregion

}

}

```

4. Шифр перестановки

Исходное сообщение разбивается на блоки длины m , где m – это длина ключа.

Ключ в шифре перестановки имеет следующий вид:

1	2	3	4
2	4	1	3

В первой строке таблицы указаны номера символов блока по порядку, а во второй строке указаны номера позиций, которые должны занимать указанные символы в зашифрованном блоке текста.

Кодирование осуществляется перестановкой букв. Таким образом первый символ из исходного блока должен быть переставлен на второе место, второй на четвертое, третий на первое, четвертый на третье.

Если данным ключом зашифровать слово кофе, то получится слово фкео.

Дешифрование производится в обратном порядке. На примере указанного ключа: второй символ из зашифрованного блока ставим на первое место, четвертый на второе, первый на третье, третий на четвертое.

При использовании любого блочного шифра (шифр перестановки не исключение), может возникнуть ситуация, когда текст не делится на равные блоки длины m . То есть остаток от деления длины текста n на длину ключа m не равен нулю.

В таких случаях длину исходного сообщения увеличивают на $m - (n \% m)$ символов, чтобы оно делилось на равные блоки длины m .

Скриншоты работы программы

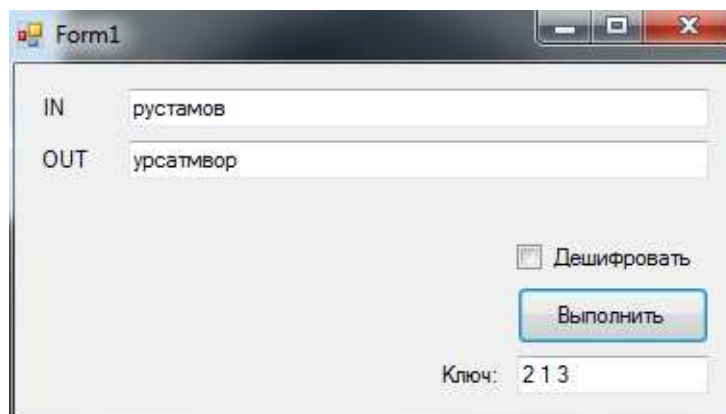


Рисунок SEQ Рисунок * ARABIC 5 - шифр перестановки

Код программы

```
// реализация класса
using System;

namespace TranspositionCipher
{
    class Transposition
    {
        private int[] key = null;

        public void SetKey(int[] _key)
        {
            key = new int[_key.Length];

            for (int i = 0; i < _key.Length; i++)
                key[i] = _key[i];
        }

        public void SetKey(string[] _key)
        {
            key = new int[_key.Length];
            for (int i = 0; i < _key.Length; i++)
                key[i] = Convert.ToInt32(_key[i]);
        }

        public void SetKey(string _key)
        { SetKey(_key.Split(' ')); }

        public string Encrypt(string input)
        {
            for (int i = 0; i < input.Length % key.Length; i++)
                input += input[i];

            string result = "";

            for (int i = 0; i < input.Length; i += key.Length)
            {
```

```

        char[] transposition = new char[key.Length];

        for (int j = 0; j < key.Length; j++)
            transposition[key[j] - 1] = input[i + j];

        for (int j = 0; j < key.Length; j++)
            result += transposition[j];
    }
    return result;
}

public string Decrypt(string input)
{
    string result = "";
    for (int i = 0; i < input.Length; i += key.Length)
    {
        char[] transposition = new char[key.Length];

        for (int j = 0; j < key.Length; j++)
            transposition[j] = input[i + key[j] - 1];

        for (int j = 0; j < key.Length; j++)
            result += transposition[j];
    }
    return result;
}
}

using System;
using System.Windows.Forms;

namespace TranspositionCipher
{
    public partial class Form1 : Form
    {
        Transposition t;

        public Form1()
        {

```

```

        InitializeComponent();

        t = new Transposition();
    }

    private void button1_Click(object sender, EventArgs e)
    {
        {
            t.SetKey(keyTextBox.Text);

            if (!cb.Checked)
                outputTextBox.Text = t.Encrypt(inputTextBox.Text);
            else
                outputTextBox.Text = t.Decrypt(inputTextBox.Text);
        }
    }
}

```

5. Шифр Виженера

Шифрование методом Виженера производится по формуле:

$$c_i = (p_i + k_i) \bmod N$$

где c_i – символ закодированного сообщения, p_i – символ исходного сообщения, k_i – символ ключа, N – мощность алфавита (количество символов в алфавите).

Символы ключа накладываются на шифруемое сообщение циклически. Например, пусть исходное сообщение: программирование на с#, а ключ = vscode, тогда на данное сообщение ключ наложится следующим образом:

п	р	о	г	р	а	м	м	и	р	о	в	а	н	и	е		н	а		с	#
v	s	c	o	d	e	v	s	c	o	d	e	v	s	c	o	d	e	v	s	c	o

Расшифровка методом Виженера производится по формуле:

$$p_i = (c_i + N - k_i) \bmod N$$

При шифровании гаммированием в качестве ключа используется последовательность символов сгенерированная с помощью генератора псевдослучайных чисел и по длине равная исходному сообщению. Псевдослучайные числа генерируются на основе заданного (всегда одинакового) начального параметра, поэтому последовательность во всех случаях получается идентичной.

Скриншоты работы программы

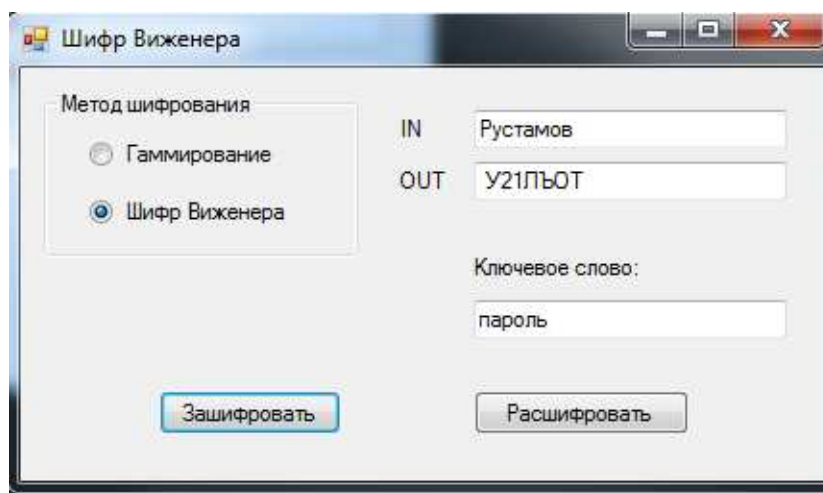


Рисунок SEQ Рисунок * ARABIC 6 - шифр Виженера

Код программы

```
using System;
using System.Windows.Forms;

namespace protect_inf_LR1
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();

            N = characters.Length;
        }

        char[] characters = new char[] { 'A', 'Б', 'В', 'Г', 'Д', 'Е', 'Ё', 'Ж',
'З', 'И',
'Й', 'К', 'Л', 'М', 'Н',
'О', 'П', 'Р', 'С',
'Т', 'У', 'Ф', 'Х', 'Ц',
'Ч', 'Ш', 'Щ', 'Ъ', 'Ы', 'Ь',
'Э', 'Ю', 'Я', ' ', '1',
'2', '3', '4', '5', '6', '7',
'8', '9', '0' };

        private int N; //длина алфавита

        //зашифровать
        private void buttonEncrypt_Click(object sender, EventArgs e)
        {
            if (radioButtonGamma.Checked)
            {
                textBox2.Text = Encode(textBox1.Text,
Generate_Pseudorandom_KeyWord(textBox1.Text.Length, 100));
            }
            else
            {
                if (textBoxKeyWord.Text.Length > 0)
                {
                    textBox2.Text = Encode(textBox1.Text, textBoxKeyWord.Text);
                }
            }
        }
    }
}
```

```

        }
        else
            MessageBox.Show("Введите ключевое слово!");
    }
}

//расшифровать
private void buttonDecipher_Click(object sender, EventArgs e)
{
    if (radioButtonGamma.Checked)
    {
        textBox2.Text = Decode(textBox1.Text,
Generate_Pseudorandom_KeyWord(textBox1.Text.Length, 100));
    }
    else
    {
        if (textBoxKeyWord.Text.Length > 0)
        {
            textBox2.Text = Decode(textBox1.Text, textBoxKeyWord.Text);
        }
        else
            MessageBox.Show("Введите ключевое слово!");
    }
}

//зашифровать
private string Encode(string input, string keyword)
{
    input = input.ToUpper();
    keyword = keyword.ToUpper();

    string result = "";

    int keyword_index = 0;

    foreach (char symbol in input)
    {
        int c = (Array.IndexOf(characters, symbol) +

```



```

        Array.IndexOf(characters, keyword[keyword_index])) % N;

        result += characters[c];
        keyword_index++;

        if ((keyword_index + 1) == keyword.Length)
            keyword_index = 0;
    }

    return result;
}

//расшифровать
private string Decode(string input, string keyword)
{
    input = input.ToUpper();
    keyword = keyword.ToUpper();

    string result = "";

    int keyword_index = 0;

    foreach (char symbol in input)
    {
        int p = (Array.IndexOf(characters, symbol) + N -
            Array.IndexOf(characters, keyword[keyword_index])) % N;

        result += characters[p];

        keyword_index++;

        if ((keyword_index + 1) == keyword.Length)
            keyword_index = 0;
    }
    return result;
}

```

```

private string Generate_Pseudorandom_KeyWord(int lenght, int startSeed)
{
    Random rand = new Random(startSeed);

    string result = "";

    for (int i = 0; i < lenght; i++)
        result += characters[rand.Next(0, characters.Length)];
    return result;
}
}
}

```

6. Шифр Хилла

Шифр Хилла является полиграммным шифром, который может использовать большие блоки с помощью линейной алгебры. Каждой букве алфавита сопоставляется число по модулю 26. Для латинского алфавита часто используется простейшая схема: A = 0, B = 1, ..., Z = 25, но это не является существенным свойством шифра. Блок из n букв рассматривается как n -мерный вектор и умножается по модулю 26 на матрицу размера $n \times n$. Если в качестве основания модуля используется число больше чем 26, то можно использовать другую числовую схему для сопоставления буквам чисел и добавить пробелы и знаки пунктуации. Элементы матрицы являются ключом. Матрица должна быть обратима, чтобы была возможна операция расшифрования.

Для $n = 3$ система может быть описана так:

$$\begin{cases} c_1 = k_{11}p_1 + k_{12}p_2 + k_{13}p_3 \pmod{26}, \\ c_2 = k_{21}p_1 + k_{22}p_2 + k_{23}p_3 \pmod{26}, \\ c_3 = k_{31}p_1 + k_{32}p_2 + k_{33}p_3 \pmod{26}, \end{cases}$$

или в матричной форме:

$$\begin{bmatrix} c_1 \\ c_2 \\ c_3 \end{bmatrix} = \begin{bmatrix} k_{11} & k_{12} & k_{13} \\ k_{21} & k_{22} & k_{23} \\ k_{31} & k_{32} & k_{33} \end{bmatrix} \begin{bmatrix} p_1 \\ p_2 \\ p_3 \end{bmatrix} \pmod{26},$$

или

$$C = KP \pmod{26},$$

где P и C — векторы-столбцы высоты 3, представляющие открытый и зашифрованный текст соответственно, K — матрица 3×3 , представляющая ключ шифрования. Операции выполняются по модулю 26.

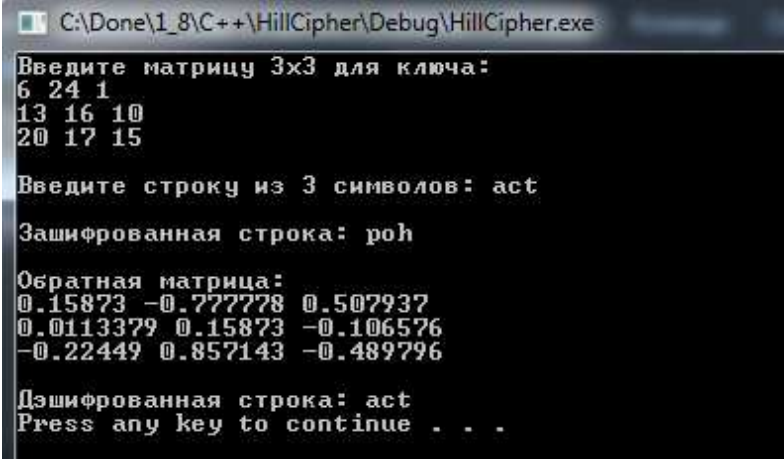
Для того, чтобы расшифровать сообщение, требуется получить обратную матрицу ключа K^{-1} . Существуют стандартные методы вычисления обратных матриц (см. способы нахождения обратной матрицы), но не все матрицы имеют обратную (см. обратная матрица). Матрица будет иметь обратную в том и только в том случае, когда её детерминант не равен нулю и не имеет общих делителей с основанием модуля. Если детерминант матрицы равен нулю или имеет общие делители с основанием модуля, то такая матрица не может использоваться в шифре Хилла, и должна быть выбрана другая матрица (в противном случае шифротекст будет невозможно расшифровать). Тем не менее, матрицы, которые удовлетворяют вышеприведенным условиям, существуют в изобилии.

В общем случае, алгоритм шифрования может быть выражен в следующем виде:

$$\text{Шифрование: } C = E(K, P) = KP \pmod{26}.$$

$$\text{Расшифрование: } P = D(K, C) = K^{-1}C \pmod{26} = K^{-1}KP \pmod{26} = P.$$

Скриншоты работы программы



```
C:\Done\1_8\C++\HillCipher\Debug\HillCipher.exe
Введите матрицу 3x3 для ключа:
6 24 1
13 16 10
20 17 15
Введите строку из 3 символов: act
Зашифрованная строка: roh
Обратная матрица:
0.15873 -0.777778 0.507937
0.0113379 0.15873 -0.106576
-0.22449 0.857143 -0.489796
Дешифрованная строка: act
Press any key to continue . . .
```

Рисунок SEQ Рисунок * ARABIC 7 - шифр Хилла

Код программы

```
#include<iostream>
#include<math.h>

using namespace std;
float encrypt[3][1], decrypt[3][1], a[3][3], b[3][3], mes[3][1], c[3][3];

void encryption(); //encrypts the message
void decryption(); //decrypts the message
void getKeyMessage(); //gets key and message from user
void inverse(); //finds inverse of key matrix

int main() {
    setlocale(LC_ALL, "Russian");
    getKeyMessage();
    encryption();
    decryption();
    system("pause");
}

void getKeyMessage() {
    int i, j;
    char msg[4];
    cout << "Введите матрицу 3x3 для ключа:\n";
    for (i = 0; i < 3; i++)
```

```

        for (j = 0; j < 3; j++) {
            cin >> a[i][j];
            c[i][j] = a[i][j];
        }
    cout << "\nВведите строку из 3 символов: ";
    cin >> msg;

    for (i = 0; i < 3; i++)
        mes[i][0] = msg[i] - 97;
}

void encryption() {
    int i, j, k;

    for (i = 0; i < 3; i++)
        for (j = 0; j < 1; j++)
            for (k = 0; k < 3; k++)
                encrypt[i][j] = encrypt[i][j] + a[i][k] * mes[k][j];

    cout << "\nЗашифрованная строка: ";
    for (i = 0; i < 3; i++)
        cout << (char)(fmod(encrypt[i][0], 26) + 97);
}

void decryption() {
    int i, j, k;
    inverse();

    for (i = 0; i < 3; i++)
        for (j = 0; j < 1; j++)
            for (k = 0; k < 3; k++)
                decrypt[i][j] = decrypt[i][j] + b[i][k] * encrypt[k][j];

    cout << "\nДешифрованная строка: ";
    for (i = 0; i < 3; i++)
        cout << (char)(fmod(decrypt[i][0], 26) + 97);
    cout << "\n";
}

```

```

void inverse() {
    int i, j, k;
    float p, q;

    for (i = 0; i < 3; i++)
        for (j = 0; j < 3; j++) {
            if (i == j)
                b[i][j] = 1;
            else
                b[i][j] = 0;
        }

    for (k = 0; k < 3; k++) {
        for (i = 0; i < 3; i++) {
            p = c[i][k];
            q = c[k][k];

            for (j = 0; j < 3; j++) {
                if (i != k) {
                    c[i][j] = c[i][j] * q - p*c[k][j];
                    b[i][j] = b[i][j] * q - p*b[k][j];
                }
            }
        }
    }

    for (i = 0; i < 3; i++)
        for (j = 0; j < 3; j++)
            b[i][j] = b[i][j] / c[i][i];

    cout << "\n\nОбратная матрица:\n";
    for (i = 0; i < 3; i++) {
        for (j = 0; j < 3; j++)
            cout << b[i][j] << " ";

        cout << "\n";
    }
}

```

}

Реализация хэш-функции (SHA-1, MD5)

Хэш-функцией называется односторонняя функция, предназначенная для получения дайджеста или "отпечатков пальцев" файла, сообщения или некоторого блока данных.

Хэш-код создается функцией H :

$$h = H(M)$$

где M является сообщением произвольной длины и h является хэш-кодом фиксированной длины.

Чтобы хэш-функция H могла использоваться в качестве аутентификатора сообщения, она должна обладать следующими свойствами:

- Хэш-функция H должна применяться к блоку данных любой длины.
- Хэш-функция H должна создавать выход фиксированной длины.
- $H(M)$ относительно легко (за полиномиальное время) вычисляется для любого значения M .
- Для любого данного значения хэш-кода h вычислительно невозможно найти M такое, что $H(M)=h$.
- Для любого данного M вычислительно невозможно найти $M' \neq M$ такое, что $H(M)=H(M')$.
- Вычислительно невозможно найти произвольную пару (M, M') такую, что $H(M)=H(M')$.

Скриншоты работы программы

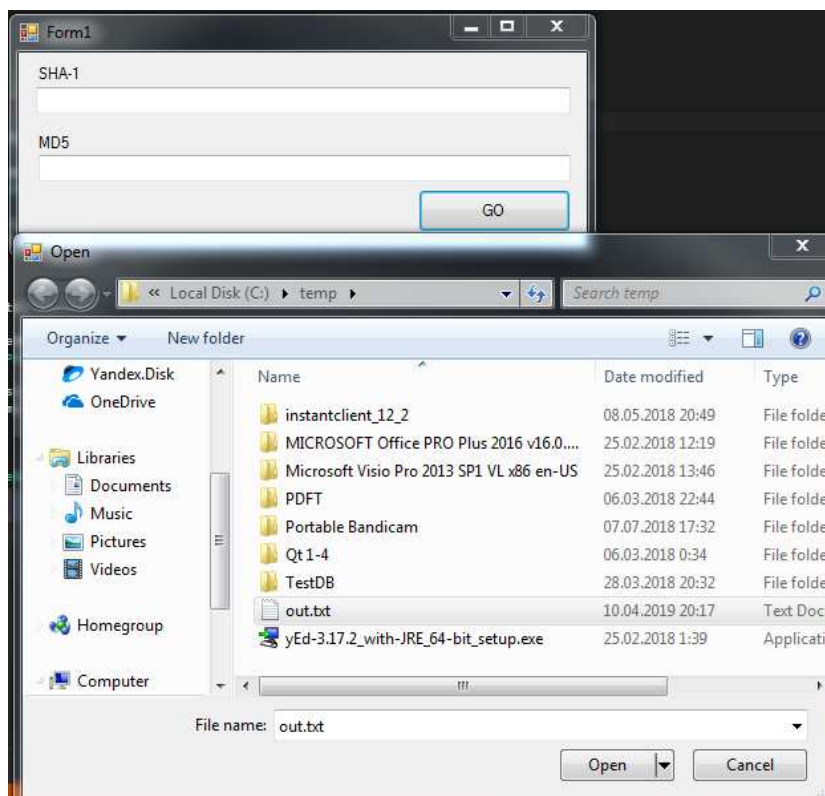


Рисунок SEQ Рисунок * ARABIC 8 - хэш функции(1)

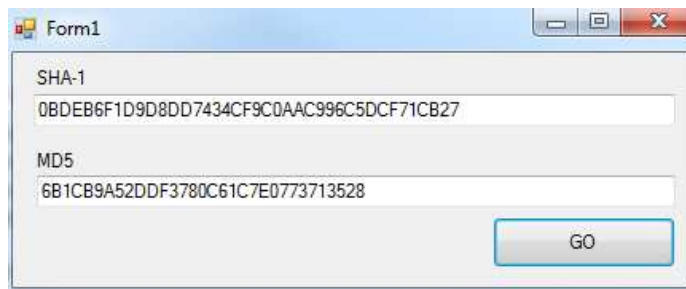


Рисунок SEQ Рисунок * ARABIC 9 - хэш функции(2)

Код программы

```
using System;
using System.IO;
using System.Windows.Forms;
using System.Security.Cryptography;

// Реализация хэш-функции (SHA-1, MD5)
namespace WindowsFormsApplication1
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
        }

        private string ComputeMD5Checksum(string path)
        {
            using (FileStream fs = System.IO.File.OpenRead(path))
            using (MD5 md5 = new MD5CryptoServiceProvider())
            {
                byte[] checkSum = md5.ComputeHash(fs);
                return BitConverter.ToString(checkSum).Replace("-", String.Empty);
            }
        }

        private string ComputeSHA1Checksum(string path)
        {
            using (FileStream fs = System.IO.File.OpenRead(path))
            using (SHA1 sha1 = new SHA1CryptoServiceProvider())
            {
                byte[] checkSum = sha1.ComputeHash(fs);
            }
        }
    }
}
```

```

        return BitConverter.ToString(checkSum).Replace("-", String.Empty);
    }
}

private void button1_Click(object sender, EventArgs e)
{
    using (OpenFileDialog openFileDialog1 = new OpenFileDialog())
    {
        if (openFileDialog1.ShowDialog() == DialogResult.OK)
        {
            string fileSelected = openFileDialog1.FileName;
            //MessageBox.Show(fileSelected);
            txtMD5.Text = ComputeMD5Checksum(fileSelected);
            txtSha1.Text = ComputeSHA1Checksum(fileSelected);
        }
    }
}
}
}
}

```

Криптосхемы асимметричной криптографии

3.1. Шифр RSA

RSA (аббревиатура от фамилий создателей: Rivest, Shamir и Adleman) – один из самых популярных алгоритмов шифрования.

Суть шифрования с открытым ключом заключается в том, что для шифрования данных используется один ключ, а для расшифрования другой (поэтому такие системы часто называют асимметричными).

Стойкость RSA основывается на большой вычислительной сложности известных алгоритмов разложения произведения простых чисел на сомножители. Например, легко найти произведение двух простых чисел 7 и 13 даже в уме – 91. Попробуйте в уме найти два простых числа, произведение которых равно 323 (числа 17 и 19). Конечно, для современной вычислительной техники найти два простых числа, произведение которых равно 323, не проблема. Поэтому для надежного шифрования алгоритмом RSA, как правило, выбираются простые числа, количество двоичных разрядов которых равно нескольким сотням.

Первым этапом любого асимметричного алгоритма является создание получателем шифрограмм пары ключей: открытого и секретного. Для алгоритма RSA этап создания ключей состоит из следующих операций.

№ п/п	Описание операции	Пример
1	Выбираются два простых числа p и q .	$p=7, q=13$
2	Вычисляется произведение $n = p * q$.	$n=91$
3	Вычисляется функция Эйлера, равная $(n)=(p-1)(q-1)=n-p-q+1$. Результат расчета данной функции равен количеству положительных чисел, не превосходящих n и взаимно простым с n .	$(n)=(7-1)(13-1)=91-7-13+1 = 72$
4	Выбирается произвольное число e ($0 < e < n$), взаимно простое с результатом функции Эйлера (e и (n)). Число e называется открытой экспонентой.	$e=5$
5	Вычисляется секретный ключ d из соотношения $(d * e) \bmod (n) = 1$. Число d называется закрытой экспонентой. Обычно пользуются выражением $de = 1 + k\phi(n)$, где k - некоторое целое число.	$(d * 5) \bmod 72 = 1$, $d = 29$
6	Публикуются открытые ключи e и n в специальном хранилище, где исключается возможность его подмены (общедоступном сертифицированном справочнике).	

Скриншоты работы программы

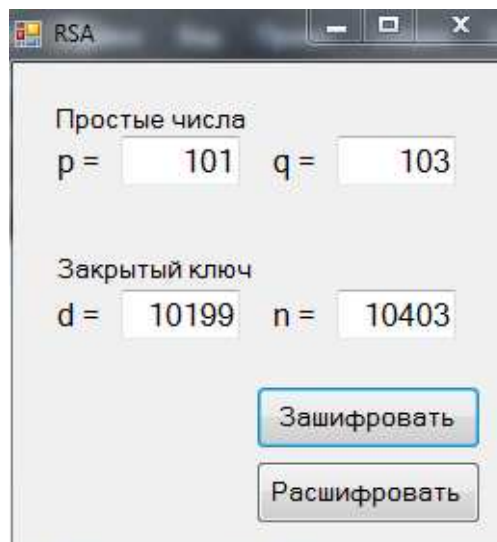


Рисунок SEQ Рисунок * ARABIC 10 - Шифр RSA(1)

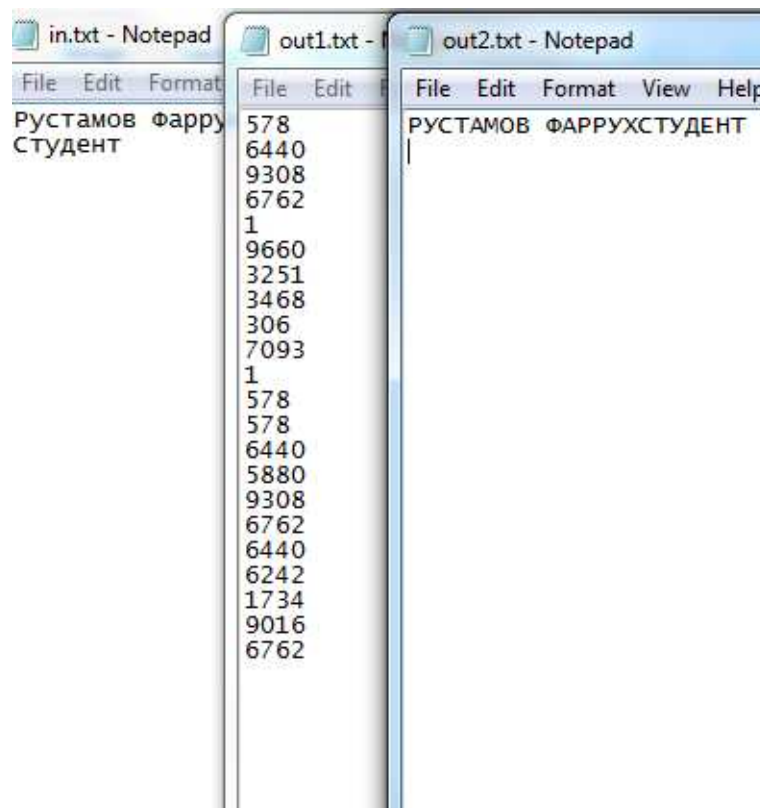


Рисунок SEQ Рисунок * ARABIC 11 - Шифр RSA(2)

Код программы

```
using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.IO;
using System.Windows.Forms;

using System.Numerics;

namespace protect_inf_LR1
{
    public partial class Form1 : Form
    {
        char[] characters = new char[] { '#', 'A', 'Б', 'В', 'Г', 'Д', 'Е', 'Ё',
        'Ж', 'З', 'И',
        'Й', 'К', 'Л', 'М', 'Н',
        'О', 'П', 'Р', 'С',
        'Т', 'У', 'Ф', 'Х', 'Ц',
        'Ч', 'Ш', 'Щ', 'Ъ', 'Ы', 'Ь',
        'Э', 'Ю', 'Я', ' ', '1',
        '2', '3', '4', '5', '6', '7',
        '8', '9', '0' };

        public Form1()
        {
            InitializeComponent();

            //зашифровать
            private void buttonEncrypt_Click(object sender, EventArgs e)
            {
                if ((textBox_p.Text.Length > 0) && (textBox_q.Text.Length > 0))
                {
                    long p = Convert.ToInt64(textBox_p.Text);
                    long q = Convert.ToInt64(textBox_q.Text);

                    if (IsTheNumberSimple(p) && IsTheNumberSimple(q))
                    {
```

```

        string s = "";

        StreamReader sr = new StreamReader("in.txt");

        while (!sr.EndOfStream)
        {
            s += sr.ReadLine();
        }
        sr.Close();
        s = s.ToUpper();
        long n = p * q;
        long m = (p - 1) * (q - 1);
        long d = Calculate_d(m);
        long e_ = Calculate_e(d, m);
        List<string> result = RSA_Endoce(s, e_, n);
        StreamWriter sw = new StreamWriter("out1.txt");
        foreach (string item in result)
            sw.WriteLine(item);
        sw.Close();

        textBox_d.Text = d.ToString();
        textBox_n.Text = n.ToString();

        Process.Start("out1.txt");
    }
    Else MessageBox.Show("p или q - не простые числа!");
}
Else MessageBox.Show("Введите p и q!");
}

//расшифровать
private void buttonDecipher_Click(object sender, EventArgs e)
{
    if ((textBox_d.Text.Length > 0) && (textBox_n.Text.Length > 0))
    {
        long d = Convert.ToInt64(textBox_d.Text);
        long n = Convert.ToInt64(textBox_n.Text);
    }
}

```

```

        List<string> input = new List<string>();

        StreamReader sr = new StreamReader("out1.txt");

        while (!sr.EndOfStream)
        {
            input.Add(sr.ReadLine());
        }

        sr.Close();

        string result = RSA_Deduce(input, d, n);

        StreamWriter sw = new StreamWriter("out2.txt");
        sw.WriteLine(result);
        sw.Close();

        Process.Start("out2.txt");
    }
    else
        MessageBox.Show("Введите секретный ключ!");
}

//проверка: простое ли число?
private bool IsTheNumberSimple(long n)
{
    if (n < 2)
        return false;

    if (n == 2)
        return true;

    for (long i = 2; i < n; i++)
        if (n % i == 0)
            return false;

    return true;
}

```

```

}

//зашифровать
private List<string> RSA_Endoce(string s, long e, long n)
{
    List<string> result = new List<string>();

    BigInteger bi;

    for (int i = 0; i < s.Length; i++)
    {
        int index = Array.IndexOf(characters, s[i]);

        bi = new BigInteger(index);
        bi = BigInteger.Pow(bi, (int)e);

        BigInteger n_ = new BigInteger((int)n);

        bi = bi % n_;

        result.Add(bi.ToString());
    }

    return result;
}

//расшифровать
private string RSA_Dedoce(List<string> input, long d, long n)
{
    string result = "";

    BigInteger bi;

    foreach (string item in input)
    {
        bi = new BigInteger(Convert.ToDouble(item));
        bi = BigInteger.Pow(bi, (int)d);
    }
}

```



```

        BigInteger n_ = new BigInteger((int)n);

        bi = bi % n_;

        int index = Convert.ToInt32(bi.ToString());

        result += characters[index].ToString();
    }

    return result;
}

//вычисление параметра d. d должно быть взаимно простым с m
private long Calculate_d(long m)
{
    long d = m - 1;

    for (long i = 2; i <= m; i++)
        if ((m % i == 0) && (d % i == 0)) //если имеют общие делители
        {
            d--;
            i = 1;
        }

    return d;
}

//вычисление параметра e
private long Calculate_e(long d, long m)
{
    long e = 10;

    while (true)
        {if ((e * d) % m == 1) break; else e++;}

    return e;
}

```

```

    }
  }
}

```

7. Шифр Эль-Гамала

Схема была предложена Тахером Эль-Гамалем в 1984 году. Он усовершенствовал систему Диффи-Хеллмана и получил два алгоритма, которые использовались для шифрования и обеспечения аутентификации. Стойкость данного алгоритма базируется на сложности решения задачи дискретного логарифмирования.

Суть задачи заключается в следующем. Имеется уравнение

$$g^x = y \bmod p.$$

Требуется по известным g , y и p найти целое неотрицательное число x (дискретный логарифм).

Порядок создания ключей приводится в следующей таблице.

№ п/п	Описание операции	Пример
1	Выбираются простое число p и два любых произвольных числа g и x меньше p (g - первообразный корень по модулю p).	$p=23, g=3, x=5$
2	Вычисляется $y = g^x \bmod p$	$y = 3^5 \bmod 23 = 243 \bmod 23 = 13$
3	Открытый ключ - y, g и p . Причем g и p можно сделать общими для группы пользователей. Закрытый ключ - x .	

Для шифрования каждого отдельного блока исходного сообщения должно выбираться случайное число k ($1 < k < p - 1$). После чего шифрограмма генерируется по следующим формулам

$$a = g^k \bmod p,$$

$$b = (y^k T) \bmod p,$$

где T – исходное сообщение;

(a, b) – зашифрованное сообщение.

Дешифрование сообщения выполняется по следующей формуле

$$T = (b (a^x)^{-1}) \bmod p$$

или

$$T = (b a^{p-1-x}) \bmod p,$$

где $(a^x)^{-1}$ – обратное значение числа a^x по модулю p .

Скриншоты работы программы

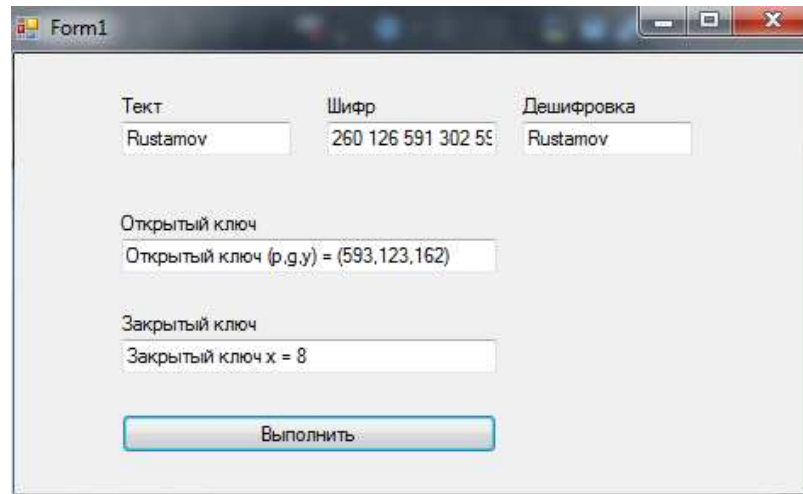


Рисунок SEQ Рисунок * ARABIC 12 - шифр Эль-Гамала

Код программы

```
using System;
using System.Windows.Forms
namespace ElGamal3
{
    public partial class Form1 : Form
    {
        public Form1() {InitializeComponent();}
        private int Rand();//Ф-я получения случайного числа
        {
            Random random = new Random();
            return random.Next();
        }
        int power(int a, int b, int n) // a^b mod n-возведение a в степень b по
        модулю n
        {
            int tmp = a; int sum = tmp;
            for (int i = 1; i < b; i++)
            {
                for (int j = 1; j < a; j++)
                {
                    sum += tmp;
                    if (sum >= n) {sum -= n; }
                }
                tmp = sum;
            } return tmp;
        }
    }
}
```

```

}
int mul(int a, int b, int n) // a*b mod n - умножение a на b по модулю n
{
    int sum = 0;
    for (int i = 0; i < b; i++)
    {
        sum += a;
        if (sum >= n)
            {sum -= n; }
    } return sum;
}
void crypt(int p, int g, int x, string strIn)
{
    txtBCrypt.Text = "";
    int y = power(g, x, p);
    txtBPublicKey.Text = "Открытый ключ (p,g,y) = (" + p + ", " + g + ", " +
y + ")";
    txtBSecretKey.Text = "Закрытый ключ x = " + x;
    if (strIn.Length > 0)
    {
        char[] temp = new char[strIn.Length - 1];
        temp = strIn.ToCharArray();
        for (int i = 0; i <= strIn.Length - 1; i++)
        {
            int m = (int)temp[i];
            if (m > 0)
            {
                int k = Rand() % (p - 2) + 1; // 1 < k < (p-1)
                int a = power(g, k, p);
                int b = mul(power(y, k, p), m, p);
                txtBCrypt.Text = txtBCrypt.Text + a + " " + b + " ";
            }
        }
    }
}
void decrypt(int p, int x, string strIn)
{
    if (strIn.Length > 0)

```

```

{
    txtBDecrypt.Text = "";
    string[] strA = strIn.Split(' ');
    if (strA.Length > 0)
    {
        for (int i = 0; i < strA.Length - 1; i += 2)
        {
            char[] a = new char[strA[i].Length];
            char[] b = new char[strA[i + 1].Length];
            int ai = 0;
            int bi = 0;
            a = strA[i].ToCharArray();
            b = strA[i + 1].ToCharArray();
            for (int j = 0; (j < a.Length); j++)
            {
                ai = ai * 10 + (int)(a[j] - 48);
            }
            for (int j = 0; (j < b.Length); j++)
            {
                bi = bi * 10 + (int)(b[j] - 48);
            }
            if ((ai != 0) && (bi != 0))
            {
                int deM = mul(bi, power(ai, p - 1 - x, p), p); //
                m=b*(a^x)^(-1)mod p =b*a^(p-1-x)mod p - трудно было найти нормальную формулу, в
                ней вся загвоздка
                char m = (char)deM;
                txtBDecrypt.Text = txtBDecrypt.Text + m;
            }
        }
    }
}

private void button1_Click(object sender, EventArgs e)
{
    int p = Convert.ToInt32(593);
    int g = Convert.ToInt32(123);
    int x = Convert.ToInt32(8);
    crypt(p, g, x, txtBIn.Text);
}

```

```
        decrypt(p, x, txtBCrypt.Text);  
    }  
}  
}
```

СПИСОК ЛИТЕРАТУРЫ

1. Шнайер Б. Прикладная криптография. Протоколы, алгоритмы, исходные тексты на языке Си – М.: ТРИУМФ, 2002. – 816 с.
2. Введение в криптографию / Под. ред. В.В. Яценко. – СПб.: Питер, 2001. – 288 с.
3. Зегжда Д.П. Основы безопасности информационных систем / Д.П. Зегжда, А.М. Ивацко. – М.: Горячая линия - Телеком, 2000. – 452 с.
4. Яковлев В.В. Информационная безопасность и защита информации в корпоративных сетях железнодорожного транспорта: Учебник для вузов ж.-д. транспорта / В.В. Яковлев, А.А. Корниенко. – М.: УМК МПС России, 2002. – 328 с.
5. National Institute of Standards and Technology (NIST). FIPS Pub 46-3 (Federal information processing standards publication): Data Encryption Standard (DES). Oct. 1999. <http://csrc.nist.gov/publications/fips/fips46-3/fips46-3.pdf>
6. Винокуров А.Ю. Традиционные криптографические алгоритмы. <http://www.enlight.ru/crypto/algorithms/algs.htm>
7. Малюк А.А. Введение в защиту информации в автоматизированных системах / А.А. Малюк, С.В. Пазинин, Н.С. Погожин. – М.: Горячая линия - Телеком, 2001. – 148 с.